

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	A. Priyanka	
Register Number	24011101001	

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING UPSTAIRS
- WALKING DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels. Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

The recommended laboratory subset is the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

The preprocessing produced the following input tensor dimensions:

- Training input tensor shape: (1680, 128, 9)
- Validation input tensor shape: (360, 128, 9)
- Testing input tensor shape: (360, 128, 9)
- Number of classes: 6
- Number of features per time step: 9
- Sequence length: 128

6. Temporal Data Visualization

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value

Sensor Signals vs. Time for Representative Windows of Different Activities

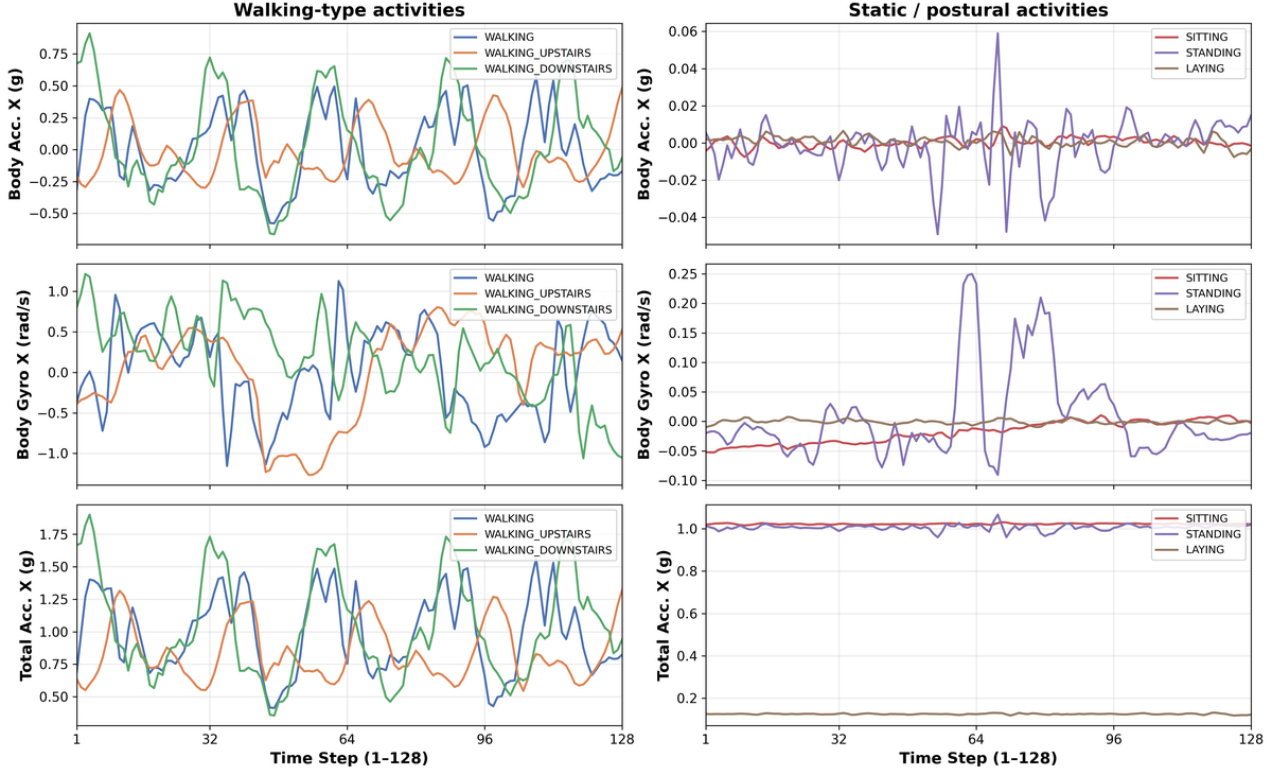


Figure 1: Temporal sensor signals for representative human activity classes.

Required inference:

1. The walking activities show clear, repeated changes in the acceleration and gyroscope signals across the 128 time steps, reflecting the movement involved in walking. In contrast SITTING, STANDING and LAYING have mostly stable signals with smaller variations although there were a few sudden spikes.
2. WALKING, WALKING UPSTAIRS and WALKING DOWNSTAIRS show similar repeated patterns in the acceleration and gyroscope signals because all three involve continuous movement. However the amount of variation is different for each activity. In comparison SITTING, STANDING and LAYING show much smaller changes in the signals and are relatively stable over time.
3. The order of the measurements matters because a single sensor value does not tell us much by itself. What is more useful is how the values change from one time step to the next. These changes and patterns over time help the RNN, LSTM and GRU understand the movement and distinguish between different activities.

7. Vanilla RNN Architecture

A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

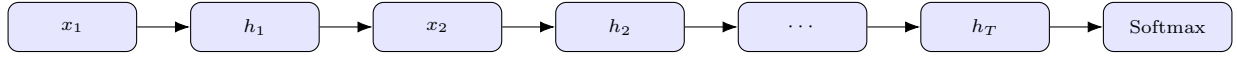
The output can be written as:

$$y_t = g(W_y h_t + b_y).$$

For sequence classification, the final hidden representation can be passed to a classifier.

The unrolled view of an RNN consists of successive hidden states:

$$x_1 \rightarrow h_1, \quad x_2 \rightarrow h_2, \quad \dots, \quad x_T \rightarrow h_T \rightarrow \text{Softmax}.$$

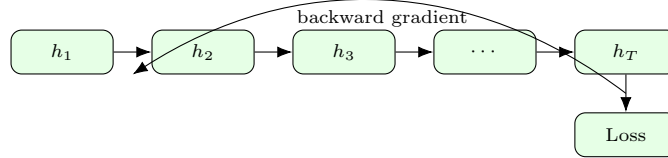


The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.

Repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.



Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as:

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0.$$

Exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider:

$$\begin{aligned} x_1 &= 0.5, & x_2 &= 0.7, & x_3 &= 0.2, \\ h_0 &= 0, & W_x &= 0.5, & W_h &= 0.8, & b &= 0.1. \end{aligned}$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

the hidden states are calculated as follows:

$$h_1 = \tanh(0.5(0.5) + 0.8(0) + 0.1) = \tanh(0.35) \approx 0.3364$$

$$h_2 = \tanh(0.5(0.7) + 0.8(0.3364) + 0.1) \approx 0.6164$$

$$h_3 = \tanh(0.5(0.2) + 0.8(0.6164) + 0.1) \approx 0.6000$$

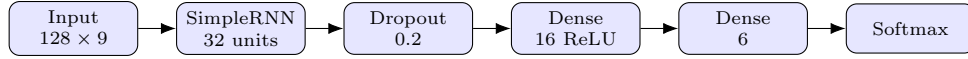
The manual calculations are compared with the corresponding program output below.

Table 1: Manual and Programmatic RNN Hidden-State Calculation

Time Step	Manual Value	Program Output
h_1	0.3364	0.3364
h_2	0.6164	0.6164
h_3	0.6000	0.6000

9. RNN Implementation

Use the following small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f).$$

The input gate is:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i).$$

The output gate is:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is:

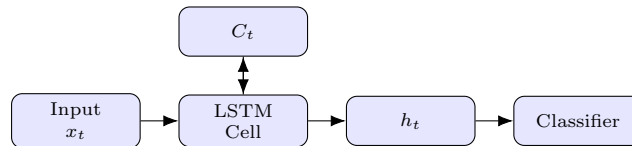
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C).$$

The cell state is:

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is:

$$h_t = o_t \tanh(C_t).$$



Required inference: The cell state works like a memory that carries useful information across the sequence. The forget gate removes information that is no longer needed. The input gate decides what new information to keep. The output gate controls what information is passed to the hidden state. Together these gates help the LSTM remember useful patterns for longer sequences.

11. GRU Architecture

GRU uses two principal gates.

The update gate is:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

and the reset gate is:

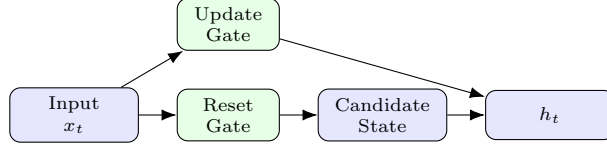
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using:

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Required inference:

GRU uses two gates called the update gate and reset gate to control the information passed through the sequence. Unlike LSTM it does not have a separate cell state and uses only a hidden state. A basic RNN also uses a hidden state but does not have gates to control the information it keeps. This makes GRU simpler than LSTM while still being able to retain useful information over longer sequences.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Table 2: Recurrent model configurations

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, the training and validation loss and accuracy were recorded for each epoch.

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

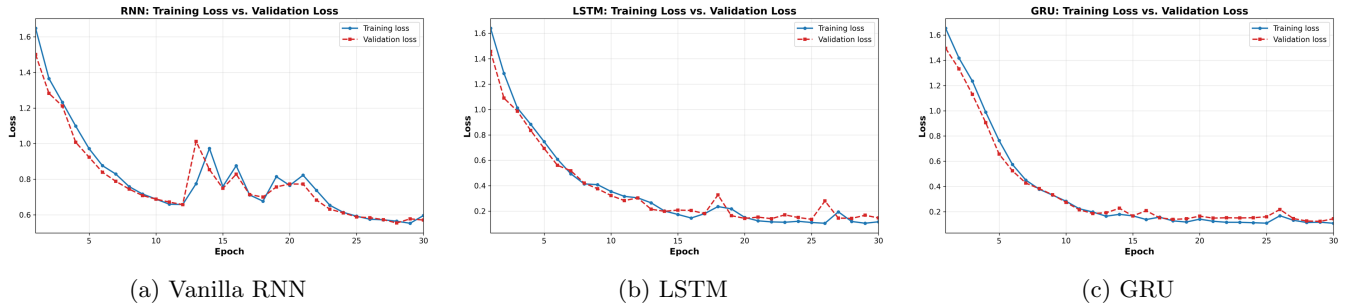


Figure 2: Training and validation loss for Vanilla RNN, LSTM and GRU.

Inference:

RNN: The training and validation loss drop during the first few epochs which shows that the model is learning. Later the curves fluctuate more and settle at a higher loss compared to LSTM and GRU. This shows that the RNN is less stable during training.

LSTM: The loss drops quickly in the early epochs and stays low towards the end. The training and validation curves remain fairly close which shows stable learning with only a small amount of overfitting.

GRU: The loss drops quickly at the beginning and then gradually levels off. The training and validation losses stay fairly close towards the end which shows that the model is learning well without a large generalization gap.

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

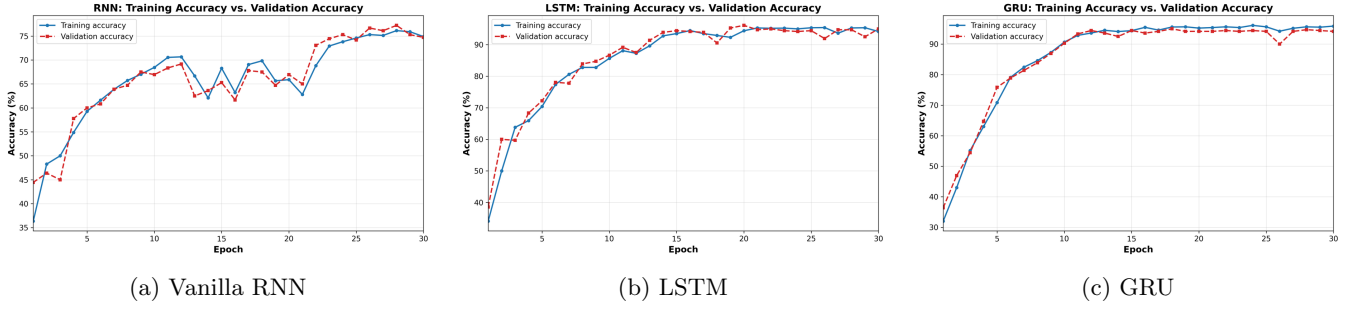


Figure 3: Training and validation accuracy for Vanilla RNN, LSTM and GRU.

Inference:

RNN: The accuracy gradually improves from the initial epochs but shows some fluctuations during training. The final validation accuracy stays around 75%, showing that the model learns the activity patterns but does not converge as well as LSTM and GRU.

LSTM: The accuracy increases steadily and reaches around 90–95% validation accuracy. The training and validation curves stay fairly close which shows stable learning with only a small generalization gap.

GRU: The accuracy improves quickly and reaches around 94–96% validation accuracy. The training accuracy approaches 100% while the validation accuracy remains stable. This shows that the model learns the training data well without a large gap between training and validation performance.

Overall observation: All three models show continuous improvement in accuracy during training and there is no clear sign of underfitting from the curves. LSTM and GRU show more stable convergence and higher validation accuracy than the Vanilla RNN.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Table 3: RNN, LSTM and GRU performance

Metric	RNN	LSTM	GRU
Accuracy (%)	77.7778	90.8333	91.6667
Macro Precision (%)	79.0454	91.0638	91.8231
Macro Recall (%)	77.7778	90.8333	91.6667
Macro F1 (%)	77.8621	90.8022	91.6143
Parameters	1974	6006	4758
Training Time (s)	24.2298	21.4384	18.0535

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

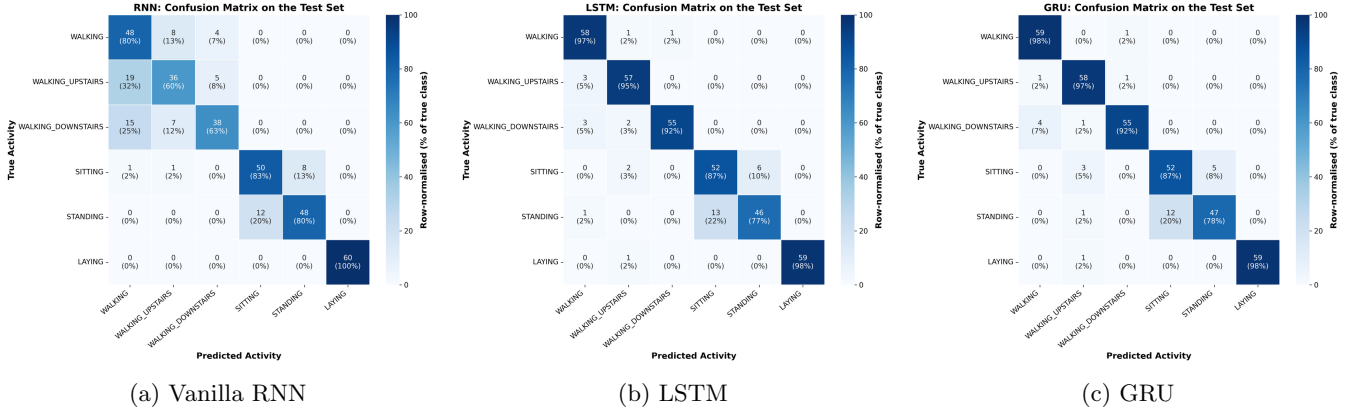


Figure 4: Confusion matrices for Vanilla RNN, LSTM and GRU on the test set.

Inference:

The confusion matrices show that most activities are classified correctly, with LAYING having one of the highest recognition rates across the models. The Vanilla RNN has more confusion between the walking activities, especially when WALKING_UPSTAIRS and WALKING_DOWNSTAIRS are predicted as WALKING.

The LSTM and GRU reduce most of these walking-related errors and have stronger diagonal values. However SITTING and STANDING are still sometimes confused in all three models since their sensor patterns are quite similar.

Overall the main errors are seen between similar walking activities and between SITTING and STANDING. The LSTM and GRU handle these confusions better than the Vanilla RNN.

16. RNN vs. LSTM vs. GRU Comparison

Table 4: Architectural comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	24.23	21.44	18.05
Test F1-score (%)	77.86	90.80	91.61

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

RNN vs. LSTM vs. GRU: Model Performance Comparison

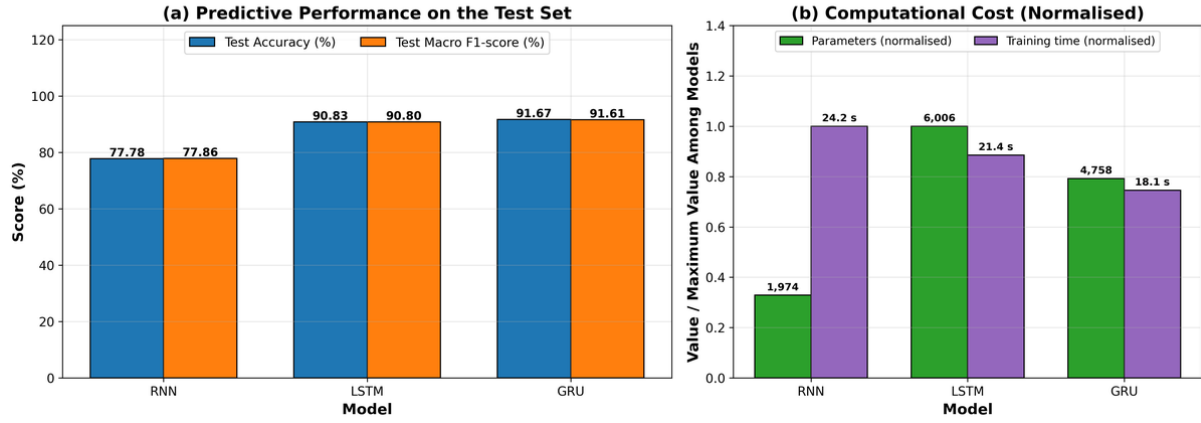


Figure 5: Comparison of test performance and computational measures for RNN, LSTM and GRU.

Students should discuss the trade-off among:

Predictive Performance ↔ Model Complexity ↔ Training Cost

Inference:

The comparison shows a clear trade off between model performance, model size and training time. The Vanilla RNN achieved 77.78% accuracy and 77.86% Macro F1-score with only 1,974 parameters. However its lower performance suggests that it has more difficulty capturing the temporal patterns in the activity sequences.

The LSTM improved the results to 90.83% accuracy and 90.80% Macro F1-score but used more parameters (6,006). The GRU performed slightly better than LSTM with 91.67% accuracy and 91.61% Macro F1-score. It also used fewer parameters (4,758) and took less time to train (18.1 seconds).

Overall the results show that adding more model complexity can improve performance, but it also increases the cost. In this experiment, the GRU gives a good balance between performance and efficiency.

Additional Exercise 1: Effect of Recurrent Units

The number of recurrent units was varied between 16, 32 and 64 to study its effect on classification performance, model size and training time. The remaining training settings were kept unchanged.

Table 5: Effect of recurrent units on model performance

Model	Units	Accuracy (%)	Macro F1 (%)	Parameters	Time (s)
RNN	16	68.06	68.04	790	22.39
RNN	32	70.56	70.25	1,974	23.38
RNN	64	47.78	47.32	5,878	23.00
LSTM	16	88.89	88.82	2,038	18.01
LSTM	32	89.72	89.70	6,006	19.38
LSTM	64	91.94	91.90	20,086	17.38
GRU	16	91.11	91.15	1,670	16.95
GRU	32	92.50	92.47	4,758	16.90
GRU	64	92.78	92.75	15,542	17.05

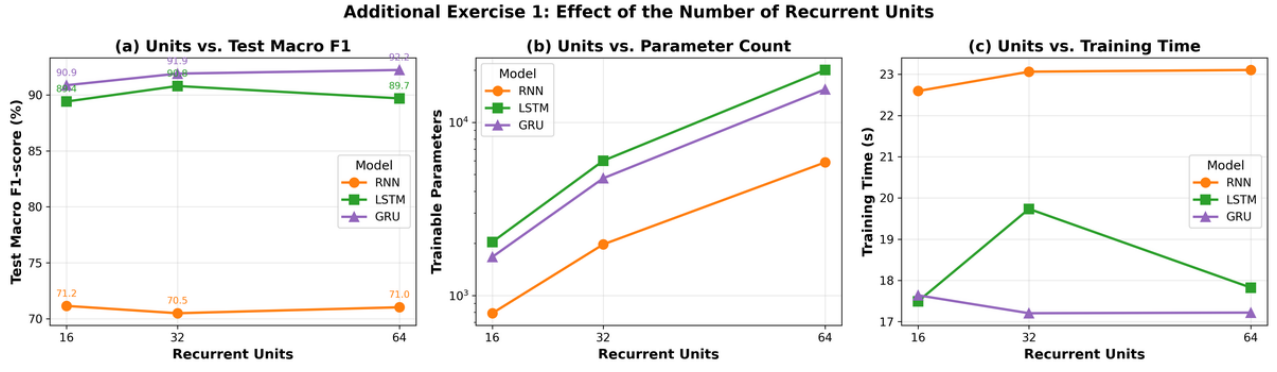


Figure 6: Effect of recurrent units on test performance, parameter count and training time.

Inference:

Increasing the number of recurrent units increased the number of parameters in all three models. LSTM and GRU generally maintained or improved their performance as the number of units increased. In contrast, the RNN performance dropped noticeably when using 64 units. Among the tested configurations, the GRU with 64 units achieved the highest Macro F1-score of 92.75%.

Additional Exercise 3: Effect of a Second Recurrent Layer

A second recurrent layer was added to the RNN, LSTM and GRU models while keeping the number of units at 32. The results were compared with the corresponding single-layer models using the same evaluation measures.

Table 6: Single-layer versus two-layer recurrent models

Model	Layers	Accuracy (%)	Macro F1 (%)	Parameters	Time (s)
RNN	1	70.56	70.25	1,974	23.38
RNN	2	80.83	80.78	4,054	35.88
LSTM	1	89.72	89.70	6,006	19.38
LSTM	2	91.39	91.31	14,326	24.36
GRU	1	92.50	92.47	4,758	16.90
GRU	2	91.11	91.11	11,094	24.60

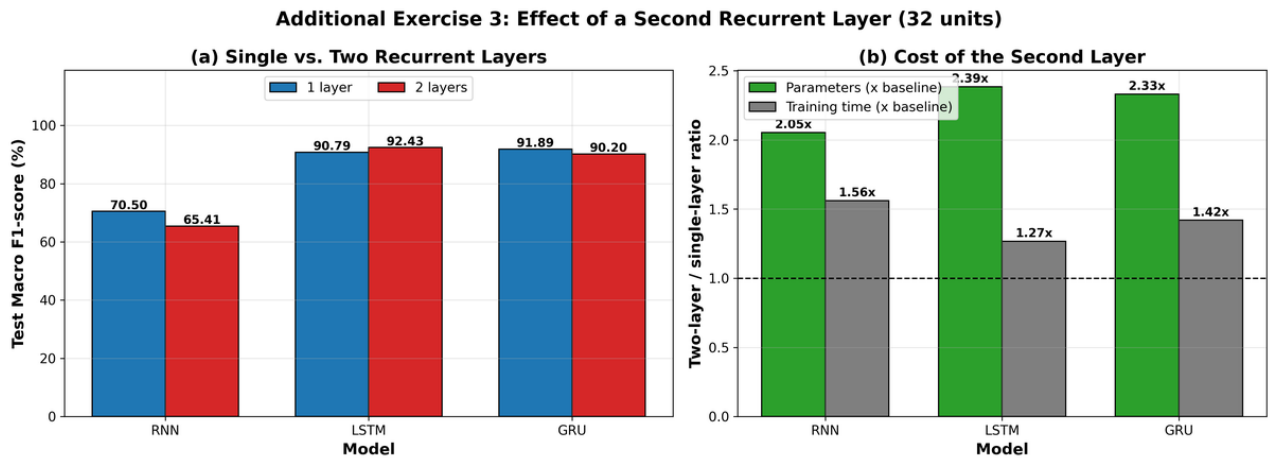


Figure 7: Comparison of single-layer and two-layer recurrent models.

Inference:

Adding a second recurrent layer improved the Macro F1-score for both the RNN and LSTM. For the GRU, however, the Macro F1-score dropped slightly after adding the extra layer. In all three models, the second layer increased the number of parameters and training time, which added to the overall computational cost.

Additional Exercise 4: Bidirectional LSTM

The unidirectional LSTM was compared with a bidirectional LSTM using 32 recurrent units. The bidirectional model processes the sequence in both forward and backward directions, allowing information from both directions to contribute to the learned representation.

Table 7: Unidirectional and bidirectional LSTM comparison

Configuration	Accuracy (%)	Macro F1 (%)	Parameters	Time (s)
LSTM(32)	89.72	89.70	6,006	19.38
Bidirectional LSTM(32)	91.94	91.90	11,894	24.57

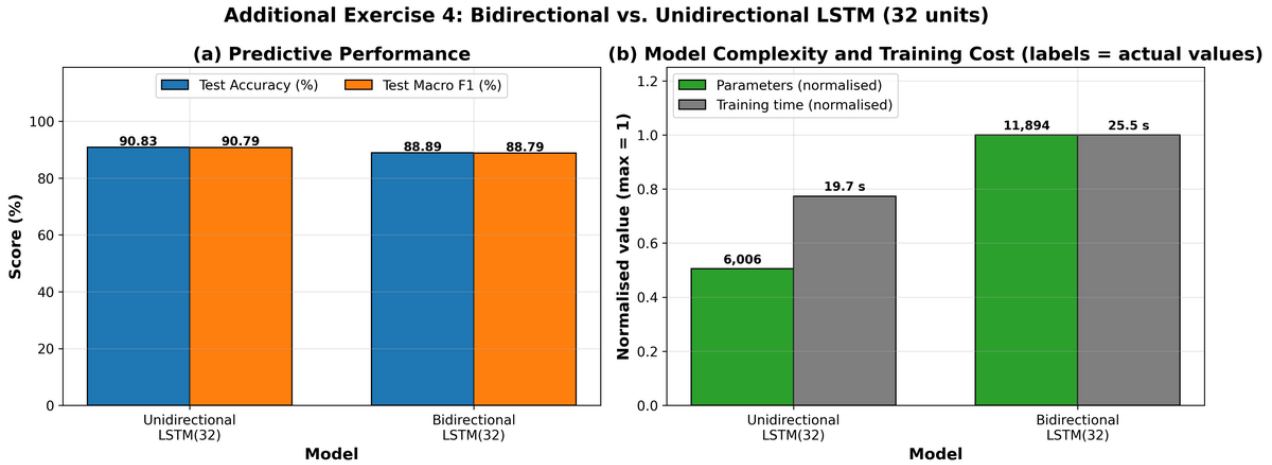


Figure 8: Comparison of unidirectional and bidirectional LSTM models.

Inference:

The bidirectional LSTM achieved a higher Macro F1-score than the unidirectional LSTM, increasing from 89.70% to 91.90%. This improvement comes with more parameters and a longer training time because the model processes the sequence in both directions.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Table 8: Effect of sequence length on test F1-score

Sequence Length	RNN F1	LSTM F1	GRU F1
32	73.98	87.49	91.38
64	74.54	90.52	92.46
128	77.86	90.80	91.61

Plot 6: Sequence Length vs. Test F1-score

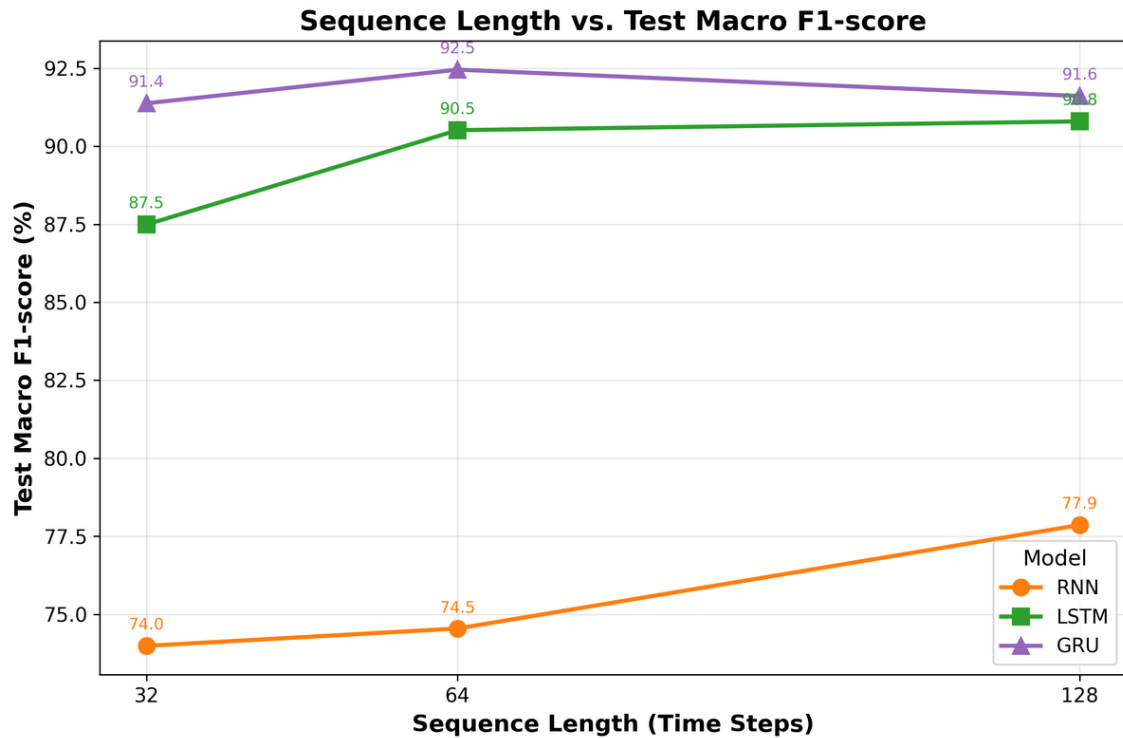


Figure 9: Effect of sequence length on test Macro F1-score for RNN, LSTM and GRU.

Inference:

The plot shows how changing the sequence length affects the test Macro F1-score of the RNN, LSTM and GRU models. A longer sequence gives the models more information about what is happening over time, but the effect is different for each model.

The RNN improves gradually from 74.0% F1-score at 32 time steps to 77.9% at 128 time steps. LSTM also improves from 87.5% to 90.8% as the sequence length increases. GRU performs well across all three lengths and reaches its highest F1-score of 92.5% at 64 time steps.

Overall, the gated models are able to capture useful temporal information even with shorter sequences. The results also show that using a longer sequence does not always improve performance, as the GRU score drops slightly at 128 time steps.

18. Video Understanding Using CNN and RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to:

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

- Predicted class: Basketball
- Actual class: Basketball
- Confidence: 0.88

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

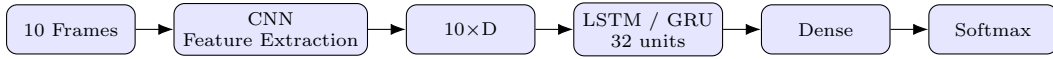
Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

21. CNN–LSTM / CNN–GRU Model

The video classification models used the following architecture:



The selected UCF101 subset contained four action classes: Basketball, Biking, TennisSwing and WalkingWithDog. A total of 120 videos were used, with 84 videos for training, 18 for validation and 18 for testing.

The pretrained MobileNetV2 network was used as a frozen feature extractor. Exactly 10 frames were uniformly sampled from each video and converted into 1280-dimensional feature vectors. Hence, each video was represented by a sequence of shape 10×1280 . The resulting feature tensors had shapes $(84, 10, 1280)$ for training, $(18, 10, 1280)$ for validation and $(18, 10, 1280)$ for testing.

Two recurrent classifiers were trained using the same extracted CNN features: an LSTM with 32 units and a GRU with 32 units.

Plot 7: Video Sample Frames

The following figure shows 10 uniformly sampled frames from a representative video.

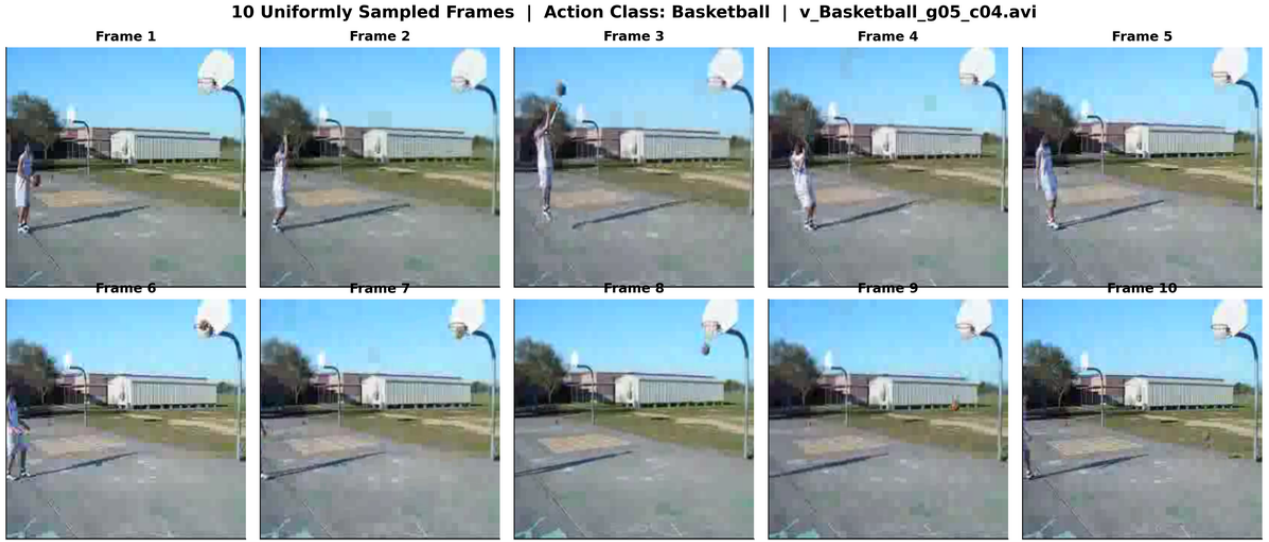


Figure 10: Ten uniformly sampled frames from a representative UCF101 video.

Inference:

The sampled frames show how the visual content changes across the video. The CNN extracts spatial information from each frame, while the recurrent network uses the ordered frame features to capture how the activity evolves over time.

Plot 8: Video Training and Validation Curves

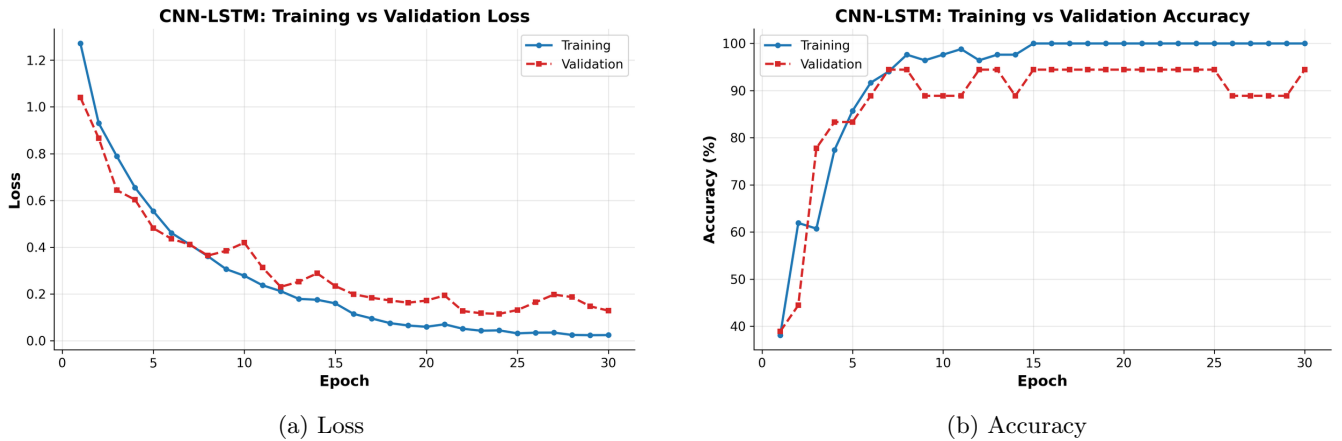
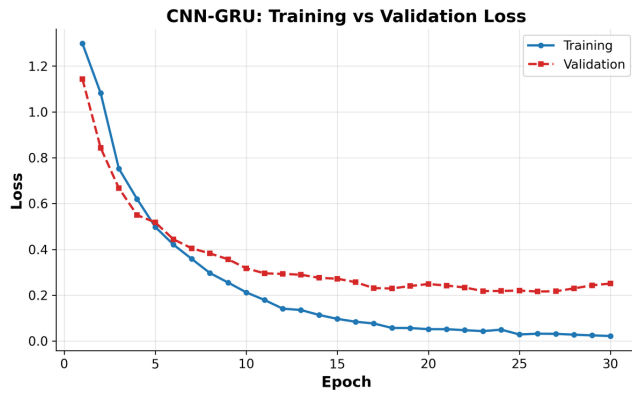


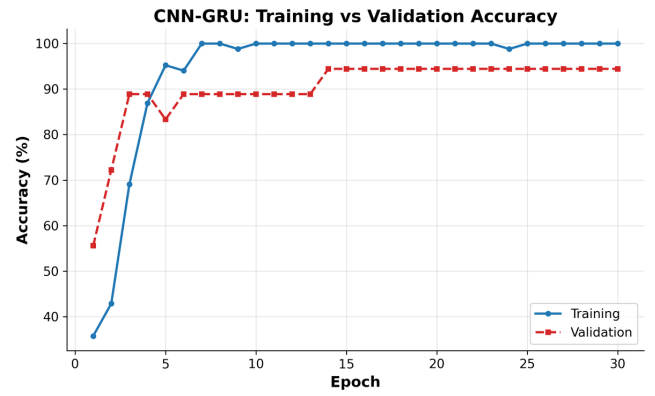
Figure 11: CNN-LSTM training and validation loss and accuracy.

Inference:

The CNN-LSTM training and validation losses decrease rapidly during the early epochs and then stabilize, while the growing gap between the curves later in training suggests that the model fits the training data more closely than the validation data. Correspondingly, training accuracy reaches 100% while validation accuracy settles around 89–94%, indicating good learning of the selected classes with some overfitting on the small dataset.



(a) Loss



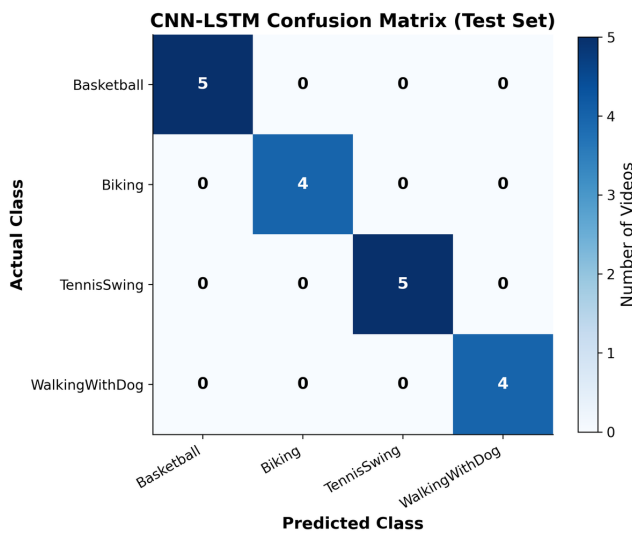
(b) Accuracy

Figure 12: CNN-GRU training and validation loss and accuracy.

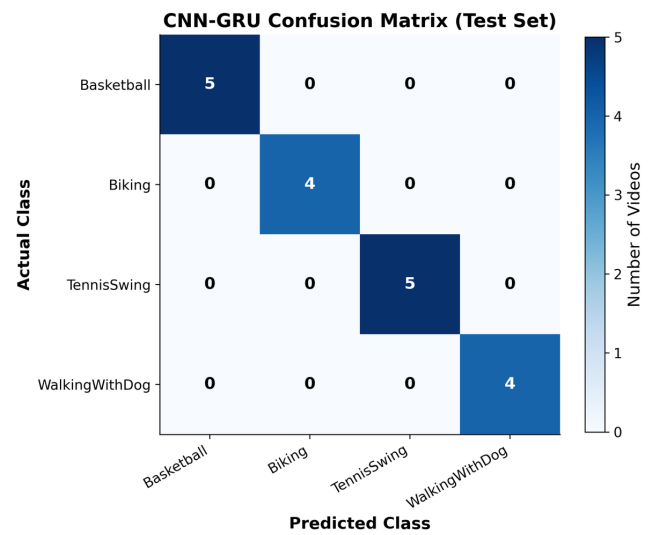
Inference:

The CNN-GRU losses decrease steadily and become stable after the initial training phase, with the lower training loss compared with validation loss toward the end showing a small generalization gap. Training accuracy reaches 100% while validation accuracy improves to about 94% and remains stable, showing effective learning of the selected video classes despite a modest training-validation gap.

Plot 9: Video Confusion Matrix



(a) CNN-LSTM



(b) CNN-GRU

Figure 13: Confusion matrices for CNN-LSTM and CNN-GRU on the test set.

Required inference:

Both the CNN-LSTM and CNN-GRU correctly classified all 18 test videos, with every entry lying on the diagonal and no misclassifications between the four selected action classes. Both models therefore achieved 100% class-wise recognition on this test subset.

Additional Exercise 6: CNN-LSTM vs CNN-GRU

Both recurrent models used identical CNN features, allowing their performance and computational requirements to be compared under the same conditions.

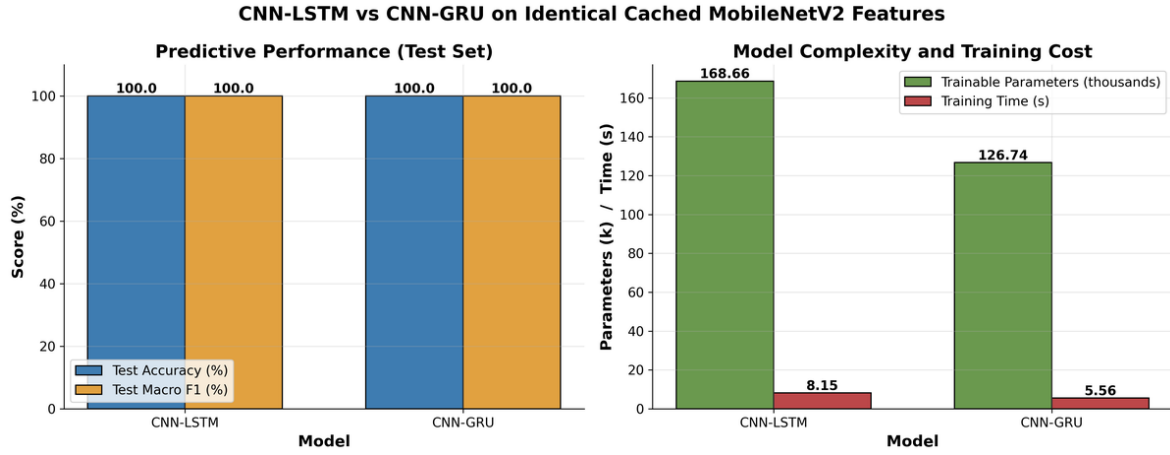


Figure 14: Comparison of CNN-LSTM and CNN-GRU performance and computational measures.

Both models achieved 100.00% test accuracy and Macro F1-score. The CNN-GRU used 126,740 trainable parameters and required 5.56 seconds of training, whereas the CNN-LSTM used 168,660 parameters and required 8.15 seconds.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.

The architecture is:



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

For the sequence-to-sequence reversal task, each input sequence contains five integer tokens and the expected output is the reversed sequence.

For example:

$$[3, 8, 1, 5, 7] \rightarrow [7, 5, 1, 8, 3].$$

Five examples from the held-out test set are shown below. The predicted outputs were generated by the trained encoder-decoder model.

Table 9: Sequence-to-sequence test examples

Sample	Input Sequence	Expected Output	Predicted Output
1	[18,12,19,13,7]	[7,13,19,12,18]	[7,13,19,12,18]
2	[12,16,19,8,17]	[17,8,19,16,12]	[17,8,19,16,12]
3	[17,14,16,14,6]	[6,14,16,14,17]	[6,14,16,14,17]
4	[9,11,4,2,6]	[6,2,4,11,9]	[6,2,4,11,9]
5	[19,6,4,11,10]	[10,11,4,6,19]	[10,11,4,6,19]

24. Sequence-to-Sequence Evaluation

The trained encoder-decoder models were evaluated on the held-out test sets using token accuracy and sequence accuracy. Training and validation losses were also recorded during training.

Table 10: Sequence-to-sequence evaluation results

Metric	Task A: Reversal (5→5)	Task B: Different Length (5→4)
Token Accuracy (%)	99.82	99.94
Sequence Accuracy (%)	99.42	99.75
Final Training Loss	0.008303	0.005644
Final Validation Loss	0.013630	0.009735
Trainable Parameters	52,373	52,373
Epochs	30	30
Training Time (s)	21.61	21.40

Training and Validation Loss

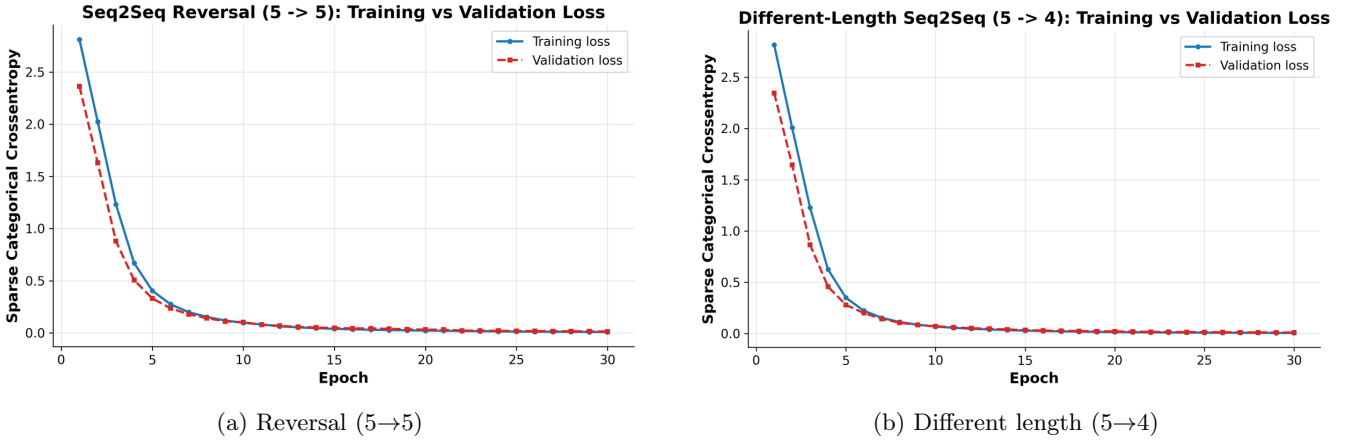


Figure 15: Training and validation loss for the sequence-to-sequence tasks.

Inference:

For both tasks, the training and validation losses decrease rapidly during the early epochs and then settle at very low values, with close final losses indicating that the encoder-decoder model learned each mapping effectively. This holds even for the different-length task, showing that the model can learn the mapping when the output sequence length differs from the input.

Training and Validation Token Accuracy

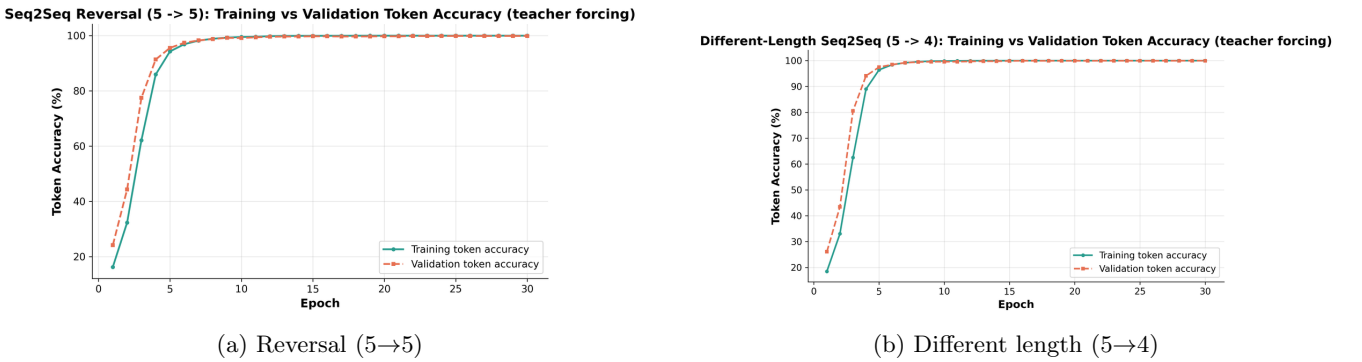


Figure 16: Training and validation token accuracy for the sequence-to-sequence tasks.

Inference:

Token accuracy rises rapidly and approaches 100% for both tasks, showing that the model learns the sequence mapping effectively. However, sequence accuracy is lower because it requires every token in the sequence to be

correct; even one incorrect token makes the entire sequence incorrect. This explains the gap between token and sequence accuracy, with 99.82% vs. 99.42% for Task A and 99.94% vs. 99.75% for Task B.

25. Consolidated Results

The final performance of the sequence-learning and video-understanding models is summarized below.

Table 11: Consolidated model results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	77.78%	79.05%	77.78%	77.86%	1,974
LSTM	90.83%	91.06%	90.83%	90.80%	6,006
GRU	91.67%	91.82%	91.67%	91.61%	4,758
CNN-LSTM	100.00%	100.00%	100.00%	100.00%	168,660
CNN-GRU	100.00%	100.00%	100.00%	100.00%	126,740

The sequence-to-sequence results are reported separately because the task uses token-level and sequence-level accuracy rather than the classification metrics used for the other models.

Table 12: Sequence-to-sequence results

Task	Token Accuracy	Sequence Accuracy	Training Loss	Validation Loss
Reversal (5→5)	99.82%	99.42%	0.008303	0.013630
Different Length (5→4)	99.94%	99.75%	0.005644	0.009735

26. Discussion Questions

1. What is a recurrent neural network?

A Recurrent Neural Network (RNN) is a neural network mainly used for sequential data. It keeps a hidden state from the previous time step and uses it along with the current input. This allows the model to use information from earlier parts of the sequence when making predictions.

2. What information is represented by the hidden state?

The hidden state contains information from the sequence that the model has processed so far. It can be thought of as the model's memory at the current time step and helps it use the previous context when processing new input.

3. Why is sequence order important?

The order of values matters because the pattern formed over time can tell us what is happening. For example, sensor readings during walking follow a particular pattern. If the readings are shuffled, that pattern is lost and the model cannot properly understand the movement.

4. Explain the difference between a feed-forward network and an RNN.

A feed-forward network processes its inputs without keeping information from previous inputs. An RNN maintains a hidden state that carries information from earlier time steps. Because of this, RNNs are more suitable for problems where the order of the data matters.

5. Explain Backpropagation Through Time.

Backpropagation Through Time (BPTT) is used to train an RNN. The network is viewed across all its time steps and the error is propagated backwards through those steps. The weights are then updated based on the errors calculated across the sequence.

6. Why can gradients vanish during BPTT?

During BPTT, the gradients are passed through many time steps. When the values being multiplied are very small, the gradient can become smaller and smaller as it moves backwards. This makes it difficult for the RNN to learn from information that occurred much earlier in the sequence.

7. Why can gradients explode during BPTT?

The opposite can also happen when the repeated multiplications make the gradients very large. This is known as the exploding gradient problem. Very large gradients can make the training unstable and cause the model weights to change too much.

8. What are long-term dependencies?

Long-term dependencies occur when information from an earlier part of a sequence is needed to understand something that happens later. In our activity recognition task, a movement pattern from earlier time steps may help the model identify the activity at later time steps.

9. **What is the role of the LSTM cell state?**
The LSTM cell state acts like a memory that carries useful information through the sequence. It allows important information to be kept for several time steps while the gates decide what should be removed or added.
10. **What are the forget, input and output gates in LSTM?**
The forget gate decides what old information should be removed. The input gate decides what new information should be stored. The output gate controls what information from the cell state is passed to the hidden state.
11. **What are the reset and update gates in GRU?**
The reset gate controls how much of the previous hidden state is used when creating new information. The update gate decides how much of the old hidden state should be kept and how much should be replaced with the new information.
12. **Compare LSTM and GRU.**
Both LSTM and GRU use gates to handle information over a sequence. LSTM has a separate cell state and hidden state along with three main gates. GRU has only a hidden state and uses two main gates. Because of this simpler structure, GRU usually has fewer parameters than LSTM.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?**
GRU combines some of the functions that are handled separately in LSTM. It also does not have a separate cell state. This makes the GRU simpler and usually gives it fewer parameters while still allowing it to learn information over time.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**
The models should be tested using the same dataset, preprocessing, data split, training settings and evaluation metrics. This makes the comparison fair and ensures that any difference in performance mainly comes from the model architecture.
15. **What does the confusion matrix reveal that accuracy alone does not?**
Accuracy only tells us the overall percentage of correct predictions. A confusion matrix gives more detail by showing how well each class is recognized and which classes are being confused with each other. This helps us find the specific areas where the model makes mistakes.
16. **Why should macro F1-score be reported for multi-class classification?**
Macro F1-score calculates the F1-score for each class separately and then takes the average. This gives all classes equal importance. It is useful for checking whether the model is doing well across all classes and not just getting a high overall accuracy.
17. **What is the significance of sequence length?**
Sequence length tells us how many time steps the model looks at together. A longer sequence gives the model more information about what is happening over time but it also takes more computation. So using a longer sequence does not always mean better performance.
18. **Why can a CNN be used as a feature extractor for video?**
A video contains a sequence of image frames and CNNs are good at finding useful visual features from images. A pretrained CNN can process each frame and convert it into a smaller feature representation. These features can then be given to an LSTM or GRU to study the sequence.
19. **What type of information is extracted by the CNN?**
The CNN extracts visual information from each frame. This can include things such as edges, shapes, textures, objects and other patterns that are useful for describing what is present in the image.
20. **What type of information is learned by the recurrent network?**
The recurrent network focuses on how the visual features change from one frame to another. This gives it information about the movement and order of events in the video, which helps it understand the overall action.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**
Each frame represents a different point in time, so the CNN features need to remain in the same order as the original video. Arranging them as a sequence allows the LSTM or GRU to learn how the visual information changes over time.
22. **Explain the encoder and decoder in sequence-to-sequence learning.**
The encoder reads the input sequence and creates a representation of the information in it. The decoder then uses this representation to produce the output sequence step by step. In our experiment, the encoder reads the input and the decoder generates the reversed sequence.
23. **What is teacher forcing?**
Teacher forcing is a training method where the decoder is given the correct previous output token instead of its own previous prediction. This helps the decoder learn the correct output sequence faster during training.
24. **Differentiate token accuracy and sequence accuracy.**
Token accuracy checks how many individual tokens are predicted correctly. Sequence accuracy is stricter because every token in the sequence must be correct for the whole sequence to be counted as correct. Because of this, sequence accuracy can be lower even when token accuracy is very high.
25. **Why must the test set remain untouched during model selection?**
The test set should be kept aside until the final evaluation. If we use it while choosing the model or tuning its settings, information from the test data can influence our decisions. Keeping it untouched gives a more reliable

idea of how the final model performs on unseen data.

27. Additional Exercises

1. Effect of recurrent units:

Changing the number of recurrent units changed both the model size and its performance. GRU-64 gave the highest Macro F1-score of 92.75% with 15,542 parameters. On the other hand RNN-64 dropped to 47.32% Macro F1-score. So adding more units did not always lead to better results for every model.

2. GRU vs. LSTM with the same number of units:

With 32 units the GRU achieved a Macro F1-score of 92.47% while the LSTM achieved 89.70%. The GRU also had fewer parameters with 4,758 compared to 6,006 for LSTM and took less time to train. This shows that the simpler GRU structure can still give strong performance with lower computational cost.

3. Effect of adding a second recurrent layer:

Adding another recurrent layer improved the RNN Macro F1-score from 70.25% to 80.78%. The LSTM also improved from 89.70% to 91.31%. For the GRU the score dropped slightly from 92.47% to 91.11%. The extra layer also increased the number of parameters and the training time in all three models. So a deeper model did not always mean better performance.

4. Bidirectional vs. unidirectional LSTM:

The bidirectional LSTM achieved a higher Macro F1-score of 91.90% compared to 89.70% for the unidirectional LSTM. However it also had more parameters which increased from 6,006 to 11,894. Its training time also increased from 19.38 to 24.57 seconds. This shows that using information from both directions can improve the result but also makes the model more expensive to train.

5. Effect of sequence length on computational cost:

The training time generally increased when the sequence length was increased. The increase was more noticeable for the 128-step sequences. A longer sequence also did not always give a better F1-score. For example the GRU performed best at 64 time steps and its F1-score dropped slightly at 128. This shows that more temporal information also comes with extra computation and does not always improve the result.

6. CNN-LSTM vs. CNN-GRU with identical CNN features:

Both models used the same frozen MobileNetV2 features and both reached 100% test accuracy and 100% Macro F1-score on the selected video data. The CNN-GRU was smaller with 126,740 parameters compared to 168,660 for CNN-LSTM. It also trained faster at 5.56 seconds compared to 8.15 seconds. So both models gave the same observed performance while the GRU used fewer resources.

7. Seq2seq with different input and output lengths:

In the original reversal task the input and output both had 5 tokens. In the modified task the input still had 5 tokens but the output had only 4 tokens. The model achieved 99.94% token accuracy and 99.75% sequence accuracy on this task. This shows that the encoder-decoder model was able to handle a case where the output sequence was shorter than the input sequence.

28. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder-decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

29. Source Code

The source code and implementation are available in this GitHub repository: **GitHub Repository**

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>